

Tutorial 6: Web Scraping Practice

1. Introduction

Web scraping is the process of collecting data from web pages automatically using programs. Many real datasets are available only through websites rather than downloadable files. This tutorial focuses on extracting tabular data from HTML pages and converting it into structured form suitable for analysis.

This tutorial emphasizes reliability. Students will work with real-world issues such as unstable connections, inconsistent HTML structure, missing elements, and character encoding problems.

Learning Outcomes

After completing this tutorial, students should be able to:

- Understand how browsers communicate with web servers
- Retrieve HTML content programmatically
- Inspect page structure to locate tabular data
- Extract and clean table contents
- Convert scraped data into DataFrame format
- Save results for later use
- Handle network errors and parsing failures
- Resolve encoding and decoding problems

2. Web Scraping Fundamentals

2.1 HTTP Request–Response Model

Web scraping relies on HTTP communication.

1. Client sends request (GET, POST)
2. Server processes request
3. Server returns response

Important response components:

- Status code
- Headers
- Body (HTML content)

Common status codes:

- **200** — Success
- **301/302** — Redirect
- **403** — Access denied
- **404** — Not found
- **500** — Server error

Understanding these helps diagnose failures.

2.2 Legal and Ethical Considerations

Before scraping:

- Check website terms of use
- Inspect robots.txt rules
- Avoid sending too many requests
- Respect data ownership

Scraping responsibly prevents blocking and legal issues.

2.3 HTML Structure Overview

Web pages use nested tags.

Typical table structure:

```
<table>

  <tr>

    <th>Header</th>

  </tr>

  <tr>

    <td>Value</td>

  </tr>

</table>
```

Hierarchy:

- table → rows
- row → cells
- cells contain data

Understanding nesting is essential for extraction.

3. Environment Setup

Install required packages:

```
pip install requests beautifulsoup4 pandas lxml
```

Import libraries:

```
import requests  
  
from bs4 import BeautifulSoup  
  
import pandas as pd
```

Library roles:

- requests — retrieve web content
- BeautifulSoup — parse HTML
- pandas — structure data

4. Retrieving Web Page Content

4.1 Basic Request

```
url = "https://example.com/table"  
  
response = requests.get(url)  
  
  
print(response.status_code)
```

If status code is 200, retrieval succeeded.

4.2 Adding Headers

Some servers block non-browser clients.

```
headers = {"User-Agent": "Mozilla/5.0"}  
  
response = requests.get(url, headers=headers)
```

Headers help mimic browser behavior.

4.3 Timeout Control

Avoid hanging requests.

```
response = requests.get(url, timeout=10)
```

Timeout ensures script responsiveness.

5. Parsing HTML with BeautifulSoup

5.1 Create Parser Object

```
soup = BeautifulSoup(response.text, "html.parser")
```

Alternative parser:

```
soup = BeautifulSoup(response.text, "lxml")
```

lxml is often faster.

5.2 Inspect Page Structure

```
print(soup.title)
```

```
print(soup.find_all("table"))
```

Use browser developer tools to identify tags.

5.3 Extract Table Rows

```
table = soup.find("table")
```

```
rows = table.find_all("tr")
```

```
dataset = []
```

```
for row in rows:
```

```
    cols = row.find_all(["th", "td"])
```

```
    cols = [c.get_text(strip=True) for c in cols]
```

```
    dataset.append(cols)
```

This creates nested list structure.

5.4 Cleaning Extracted Data

Remove unwanted characters:

```
cleaned = [  
    [item.replace("\n", "") for item in row]  
    for row in dataset  
]
```

Data cleaning improves usability.

6. Converting to Structured Format

6.1 Create DataFrame

```
df = pd.DataFrame(cleaned[1:], columns=cleaned[0])  
print(df.head())
```

This prepares data for analysis.

6.2 Export Options

```
df.to_csv("table.csv", index=False)  
df.to_excel("table.xlsx", index=False)
```

Exporting enables reuse.

7. Handling Exceptions

Real scraping rarely works perfectly.

7.1 Network Errors

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
except requests.exceptions.Timeout:
    print("Timeout error")
except requests.exceptions.HTTPError:
    print("HTTP error")
except requests.exceptions.RequestException as e:
    print("Other error:", e)
```

7.2 Missing Elements

```
table = soup.find("table")

if table is None:
    print("No table found")
```

7.3 Rate Limiting Awareness

```
import time

time.sleep(2)
```

8. Handling Encoding Issues

Web pages use different character sets.

8.1 Automatic Detection

```
response.encoding = response.apparent_encoding
```

```
html = response.text
```

8.2 Manual Decoding

```
html = response.content.decode("utf-8", errors="replace")
```

8.3 Removing Problem Characters

```
df = df.applymap(  
    lambda x: x.encode("ascii", "ignore").decode()  
)
```

Ensures compatibility.

Exercises

1. Install required libraries and verify that imports run without errors.
2. Send a GET request to a public webpage and print the response status code.
3. Print the first portion of the HTML content returned by the server.
4. Parse the page and count how many tables exist.
5. Extract the page title and display it.
6. Retrieve all hyperlinks from the page.
7. Extract one table and store rows in a Python list.
8. Separate table headers from data rows.
9. Convert extracted data into a pandas DataFrame.
10. Save the DataFrame to CSV format.
11. Clean whitespace and newline characters from all cells.
12. Convert numeric columns to appropriate numeric types.
13. Extract two tables from the same page and store them separately.
14. Merge two scraped tables into one dataset.
15. Implement error handling for failed requests using try-except.
16. Detect and handle missing table elements gracefully.
17. Detect encoding automatically and decode page content correctly.
18. Add delay between repeated requests to avoid server overload.
19. Write a reusable function that accepts a URL and returns extracted table data.
20. Write a short reflection describing limitations, ethical concerns, and challenges encountered during scraping.